P.PORTO

ESCOLA SUPERIOR DE TECNOLOGIA E GESTÃO

XML Schema

Bruno Oliveira (bmo@estg.ipp.pt)

PEI Processamento Estruturado de Informação

Agenda

- XSD – XML Schema
  - Elementos
  - Atributos
  - Restrições
  - Tipos de dados
- Namespaces

Introdução

- Mesmo estando bem formados, os documentos podem conter erros de validação considerando a âmbito em que estão inseridos.
- Um Schema XML descreve a estrutura e o vocabulário de um documento XML;
- O principal propósito passa por definir as regras de construção de um documento XML:
  - Os elementos e atributos que podem surgir no documento;
  - O número e a ordem dos elementos filho;
  - Tipos de dados para elementos e atributos;
  - (...)

Introdução

- Através da utilização de um parser, é possível validar documentos XML;
- Além disso, o XSD é escrito utilizado XML;
- Com XSD, Podemos:
  - Reutilizar os schemas;
  - Criar os nossos próprios tipos de dados;
  - Referenciar múltiplos schemas num documento

Introdução

- O elemento schema é o root de qualquer documento XSD;
- O elemento schema contém a declaração do namespace XSD:

```
<?xml version="1.0“ encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">

…
</xs:schema>
```

- Documentos XSD têm a extensão .XSD.

Namespaces

- O fragmento: xmlns:xs="http://www.w3.org/2001/XMLSchema"
- Indica que os elementos e tipos de dados do schema são definidos no namespace: http://www.w3.org/2001/XMLSchema
- Também indica que os elementos e tipos de dados desse namespace devem ser precedidos do prefixo: xs;

Simple elements

- Um elemento simples é um elemento XML que contém apenas texto;
- Não pode conter outros elementos ou atributos.
- No entanto, o “texto” pode ser de muitos tipos diferentes incluídos na definição do XML Schema (booleano, string, data, etc.), ou pode ser um tipo personalizado.
- Podem ser adicionadas restrições (facetas) para limitar seu conteúdo, o que faz com que os dados correspondam a um padrão específico.

Simple elements

- Exemplo:

```
(...)
<xs:element name= "nome" type= "xs:string"/>
(...)
```

Simple elements

- O XSD possui vários tipos de dados. Os tipos mais comuns são:

Tipos de dados	Exemplo (XSD)	Exemplo (XML)
xs:string	`<xs:element name="cliente" type="xs:string"/>`	`<cliente>Augusto Pinto</cliente>`
xs:decimal	`<xs:element name="preco" type="xs:decimal"/>`	`<preco>25.38</preco>`
xs:integer	`<xs:element name="idade" type="xs:integer"/>`	`<idade>25</idade>`
xs:boolean	`<xs:attribute name="stock" type="xs:boolean"/>`	`<livro stock="true">Os Maias</livro>`
xs:date	`<xs:element name="data" type="xs:date"/>`	`<data>2021-10-13</data>`
xs:time	`<xs:element name="inicio" type="xs:time"/>`	`<inicio>09:23:40</inicio>`
xs:dateTime	`<xs:element name="data_inicio" type="xs:dateTime"/>`	`<data_inicio>2021-10-13T09:23:40</data_inicio>`

Simple elements – Facets

- Um valor fixo é atribuído ao elemento e mais nenhum valor pode ser especificado:

```
(…)
<xs:element name=“cor" type="xs:string" fixed=“vermelho"/>
(…)
```

Simple elements – Facets

- Restringir o valor da “idade" para que esteja contido no intervalo de 0 e 120:

```
(...)
<xs:element name=“idade">
 <xs:simpleType>
   <xs:restriction base="xs:integer">
     <xs:minInclusive value="0"/>
     <xs:maxInclusive value="120"/>
   </xs:restriction>
 </xs:simpleType>
</xs:element>
(...)
```

Simple elements – Facets

- Para limitar o conteúdo de um elemento XML a uma série de números ou letras, podemos utilizar expressões regulares;
- Exemplo: Aceitar apenas letras minúsculas de a a z:

```
(...)
<xs:element name="nota">
  <xs:simpleType>
    <xs:restriction base="xs:string">
      <xs:pattern value="[a-z]"/>
    </xs:restriction>
  </xs:simpleType>
</xs:element>
(...)
```

Simple elements – Facets

- Exemplo: 3 letras maiúsculas de a a z:

```
(…)
  <xs:restriction base="xs:string">
   <xs:pattern value="[A-Z][A-Z][A-Z]"/>
  </xs:restriction>
(…)
```

- Exemplo: 3 letras maiúsculas ou minúsculas de a a z:

```
(…)
<xs:pattern value="[a-zA-Z][a-zA-Z][a-zA-Z]"/>
(…)
```

- Exemplo: As letras x, y e z são as únicas permitidas:

```
(…)
<xs:pattern value="[xyz]"/>
(…)
```

Simple elements – Facets

- Exemplo: 5 dígitos de 0 a 9:

```
(...)
<xs:pattern value="[0-9][0-9][0-9][0-9][0-9]"/>
(...)
```

- Exemplo: 0 ou mais ocorrências de letras de a a z:

```
(...)
<xs:pattern value="([a-z])*"/>
(...)
```

- Exemplo: Pares de letras com minúscula e maiúscula:

```
(...)
<xs:pattern value="([a-z][A-Z])+)"/>
(...)
```

Simple elements – Facets

- Exemplo: Conjunto de valores:

```
(...)
<xs:pattern value="male|female"/>
(...)
```

- Exemplo: 8 caracteres que podem ser minúsculas ou maiúsculas de a a z ou número de 0 a 9:

```
(...)
<xs:pattern value="[a-zA-Z0-9]{8}"/>
(...)
```

Simple elements – Facets

impõe que o domínio deve ter um domínio que consiste em pelo menos duas letras (por exemplo, .com, .org, etc.). O {2,} significa que deve ter pelo menos dois caracteres, mas pode ser mais longo (por exemplo, .info, .museum).

- Exemplo: Validação de um email:

```
<xs:pattern value="([A-Za-z0-9._%+-]+)@([A-Za-z0-9.-]+\.[A-Za-z]{2,})"></xs:pattern>
```

Permite um ou mais dos seguintes caracteres na parte local (antes do @):
•Letras maiúsculas e minúsculas (A-Za-z)
•Dígitos (0-9)
•Pontos (.), sublinhados (_), sinais de percentagem (%), sinais de mais (+) e hífens (-), todos eles válidos em endereços de e-mail.

O símbolo literal @

Permite um ou mais caracteres na parte do domínio (após o @):
•Letras maiúsculas e minúsculas (A-Za-z)
•Dígitos (0-9)
•Pontos (.) e hífens (-) são permitidos nos nomes de domínio.

Simple elements – Facets

- Para limitar o tamanho de um valor, podemos utilizar as restrições: maxLength e minLength;

```
(...)
<xs:length value="8"/>
(...)
```

- Exemplo: mínimo de 5 e máximo de 8 caracteres

```
(...)
<xs:minLength value="5"/>
<xs:maxLength value="8"/>
(...)
```

Complex Elements - Atributos

- Se um elemento contém atributos, é considerado um elemento complexo. No entanto, o atributo é declarado como um tipo simples;

```
<xs:attribute name=“lang" type="xs:string"/>
```

- Valores fixos ou por defeito podem também ser configurados:

```
(...)
<xs:attribute name=“title" type="xs:string" default="EN"/>
(...)

(...)
<xs:attribute name=“title" type="xs:string" fixed="EN"/>
(...)
```

Complex Elements - Atributos

Os atributos são opcionais por defeito. Para que sejam obrigatórios, é necessário utilizar o atributo “use”:

```
(…)
<xs:attribute name="lang" type="xs:string" use="required"/>
(…)
```

Complex Elements

Exemplo completo com atributos:

```
<xs:complexType name="emailType">
    <xs:sequence>
      <xs:element name="to" type="xs:string" minOccurs="1" maxOccurs="1"/>
     (…)
     <xs:element name="body" type="xs:string" minOccurs="1" maxOccurs="1"/>
    </xs:sequence>

    <xs:attribute ref="type" use="required“/>

  </xs:complexType>
```

Complex Elements

Um elemento complexo é um elemento XML que contém outros elementos e/ou atributos;

Existem 4 tipos:

- Elementos vazios: `<elementoVazio/>`
- Elementos que contêm outros elementos:

```
<pai>
    <filho1></filho1>
    <filho2></filho2>
</pai>
```

- Elementos que apenas contêm texto:

```
<elementoTexto atributo="valor">Algum texto aqui</elementoTexto>
```

- Elementos com elementos e texto.

```
<elementoComplexo>Texto aqui.<filho></filho></elementoComplexo>
```

Complex Elements

Duas formas de construir elementos complexos:

1. Na especificação do elemento:

```
(...)
<xs:element name="pessoa">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="primeiroNome" type="xs:string"/>
      <xs:element name="ultimoNome" type="xs:string"/>
    </xs:sequence>
  </xs:complexType>
</xs:element>
(...)
```

Indicador de ordem (preserva a ordem dos elementos)

Apenas o elemento pessoa pode utilizar este elemento complexo

Complex Elements

Duas formas de construir elementos complexos:

2. Utilizar o atributo type que refere o elemento que utiliza o elemento complexo:

```
(...)
<xs:element name="pessoa" type="infoPessoa"/>

<xs:complexType name=" infoPessoa">
  <xs:sequence>
    <xs:element name="primeiroNome" type="xs:string"/>
    <xs:element name="ultimoNome" type="xs:string"/>
  </xs:sequence>
</xs:complexType>
(...)
```

Vários elementos podem utilizar este elemento complexo

Complex Elements

- Um tipo complexo misto pode conter atributos, elementos e texto:

```
(...)
<xs:element name="letter" type="lettertype"/>

<xs:complexType name="lettertype" mixed="true">
  <xs:sequence>
    <xs:element name="name" type="xs:string"/>
    <xs:element name="orderid" type="xs:positiveInteger"/>
    <xs:element name="shipdate" type="xs:date"/>
  </xs:sequence>
</xs:complexType>
(...)
```

```
<letter>
  Dear Mr.<name>John Smith</name>.
  Your order <orderid>1032</orderid>
  will be shipped on <shipdate>2001-07-13</shipdate>
</letter>
```

Complex Elements: xs:simpleContent e xs:extension

- O elemento simpleContent contém extensões num tipo complexo de texto ou num tipo simples, não contendo elementos;
- Tem como elemento pai: xs:complexType;
- O elemento extension estende um tipo existente (simpleType ou complexType);
- Tem como elementos pai: xs:simpleContent ou xs:complexContent;

Complex Elements: xs:simpleContent e xs:extension

- Extensão de um elemento:

XML

```
(...)
<email type = "Profissional">xpt@gmail.com</email>
(...)
```

XSD

```
(...)
<xs:element name="email">
    <xs:complexType>
      <xs:simpleContent>
        <xs:extension base="emailType">
          <xs:attribute name="type" type= "xs:string" />
        </xs:extension>
      </xs:simpleContent>
    </xs:complexType>
  </xs:element>
(...)
```

Extension do elemento emailType

Complex Elements: xs:simpleContent e xs:extension

- Exemplo de um elemento que apenas contém texto e atributos:

```
(...)
<xs:element name="shoesize" type="shoetype"/>

<xs:complexType name="shoetype">
  <xs:simpleContent>
    <xs:extension base="xs:string">
      <xs:attribute name="country" type="xs:string" />
    </xs:extension>
  </xs:simpleContent>
</xs:complexType>
(...)
```

Complex Elements: Indicadores

- Os indicadores de ordem permitem restringir a ordem com que os elementos surgem no documento;
- Indicadores de ordem:
  - All
  - Choice
  - Sequence
- Indicadores de ocorrência:
  - maxOccurs
  - minOccurs

Complex Elements: Indicadores

- O indicador: <all> especifica que os elementos filho podem surgir em qualquer ordem mas cada um deve surgir apenas uma vez:

```
(...)
<xs:element name="pessoa">
  <xs:complexType>
    <xs:all>
      <xs:element name="primeiroNome" type="xs:string"/>
      <xs:element name="ultimoNome" type="xs:string"/>
    </xs:all>
  </xs:complexType>
</xs:element>
(...)
```

Complex Elements: Indicadores

- O indicador: <choice> especifica que qualquer um dos elementos pode ocorrer:

```
(…)
<xs:choice>
    <xs:element name=“cliente" type=“cliente"/>
    <xs:element name=“socio" type=“socio"/>
</xs:choice>
(…)
```

Complex Elements: Indicadores

- O indicador: <sequence> especifica que os elementos filho devem surgir numa ordem especifica:

```
(...)
<xs:element name="pessoa">
  <xs:complexType>
   <xs:sequence>
    <xs:element name="primeiroNome" type="xs:string"/>
    <xs:element name="ultimoNome" type="xs:string"/>
   </xs:sequence>
  </xs:complexType>
</xs:element>
(...)
```

Complex Elements: Indicadores

- Os indicadores de ocorrência determinam o número de ocasiões que um elemento pode ocorrer;
- O indicador: <maxOccurs> determina o número máximo de ocorrências:

```
(…)
<xs:sequence>
    <xs:element name=“nomeCompleto" type="xs:string"/>
    <xs:element name=“nomeFilho" type="xs:string" maxOccurs="10"/>
</xs:sequence>
(…)
```

Complex Elements: Indicadores

- O indicador: <minOccurs> determina o número mínimo de ocorrências:

```
(…)
<xs:sequence>
    <xs:element name=“nomeCompleto" type="xs:string"/>
    <xs:element name=“nomeFilho" type="xs:string" minOccurs=”0"/>
</xs:sequence>
(…)
```

Complex Elements: Indicadores

- A configuração: maxOccurs="unbounded" permite que o elemento ocorra infinitamente.

```
(…)
<xs:sequence>
    <xs:element name=“nomeCompleto" type="xs:string"/>
    <xs:element name=“nomeFilho" type="xs:string" maxOccurs=”unbounded"/>
</xs:sequence>
(…)
```

Associação de um documento XSD a um documento XML

```
<?xml version="1.0" encoding="UTF-8"?>
<pessoas xmlns:xsi=http://www.w3.org/2001/XMLSchema-instance
    xsi:noNamespaceSchemaLocation =”pessoaTypes.xsd">
  <pessoa>
    <primeiroNome>Pedro</primeiroNome>
    <ultimoNome>Pinto</ultimoNome>
  </pessoa>
  <pessoa>
    <primeiroNome>Fernando</primeiroNome>
    <ultimoNome>Santos</ultimoNome>
</pessoa>
</pessoas>
```

Referenciar elementos e tipos

- Podemos também definir tipos, que possam ser reutilizados e, de seguida, invocá-los através do atributo “type”.

Referenciar tipos

```xml
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">

    <xs:complexType name="produtoTipo">
        <xs:sequence>
            <xs:element name="nome" type="xs:string" />
            <xs:element name="quantidade" type="xs:positiveInteger" />
            <xs:element name="preco" type="xs:decimal" />
        </xs:sequence>
    </xs:complexType>

    <xs:element name="fatura">
        <xs:complexType>
            <xs:sequence>
                <xs:element name="numero_fatura" type="xs:positiveInteger" />
                <xs:element name="produto" maxOccurs="unbounded" type="produtoTipo" />
            </xs:sequence>
        </xs:complexType>
    </xs:element>

</xs:schema>
```

Modularização de um XSD em vários ficheiros

- Exemplo:

Fatura.xsd

```xml
<?xml version="1.0" encoding="UTF-8" ?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
<xs:include schemaLocation="produto.xsd"/>

<xs:element name="fatura">
    <xs:complexType>
        <xs:sequence>
          <xs:element name="numero_fatura" type="xs:positiveInteger"/>
          <xs:element name="produto" maxOccurs="unbounded" type="produtoTipo"/>
        </xs:sequence>
      </xs:complexType>
</xs:element>

</xs:schema>
```

Produto.xsd

```xml
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">

    <xs:complexType name="produtoTipo">
        <xs:sequence>
            <xs:element name="nome" type="xs:string"/>
            <xs:element name="quantidade" type="xs:positiveInteger"/>
            <xs:element name="preco" type="xs:decimal"/>
        </xs:sequence>
    </xs:complexType>

</xs:schema>
```

Namespaces

- Até agora aprendemos a criar documentos XML e os respetivos documentos de regras que nos permitem definir o vocabulário admissível para a construção de documentos;
- Considere que ficou encarregue de criar os documentos XSD para definir o conceito de ingrediente, enquanto que outro colega ficou responsável pela definição dos conceitos relacionados com uma Pizza;
- Considere ainda que o nome do ingrediente tem no máximo 50 caracteres e o nome da pizza 30 caracteres.

Namespaces

- Elementos com o mesmo nome!

Ingrediente.XSD

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema
xmlns:xs="http://www.w3.org/2001/XMLSchema">
   <xs:simpleType name="tipoNome">
      <xs:restriction base="xs:string">
         <xs:maxLength value="50" />
      </xs:restriction>
   </xs:simpleType>
(...)
</xs:schema>
```

Pizza.XSD

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema
xmlns:xs="http://www.w3.org/2001/XMLSchema">
   <xs:simpleType name=" tipoNome ">
      <xs:restriction base="xs:string">
         <xs:maxLength value="30" />
      </xs:restriction>
   </xs:simpleType>
(...)
</xs:schema>
```

Namespaces

- O que irá originar o seguinte erro:

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema
xmlns:xs="http://www.w3.org/2001/XMLSchema">
   <xs:include schemaLocation="Ingrediente.xsd"/>
   (…)
   <xs:element name="pizza">
       <xs:complexType>
          <xs:sequence>
             <xs:element name="nome" type="tipoNome" />
          </xs:sequence>
       </xs:complexType>
    </xs:element>
</xs:schema>
```

A schema cannot contain two global components with the same name; this schema contains two occurrences of ',name'.

Utilização de namespaces

- A solução para o problema apresentado nos slides anteriores passa por agrupar um conjunto de elementos de cada documento no seu próprio “espaço”;
- Isto significa que a sua referenciação passa a ser feita considerando o espaço ao qual cada elemento reside;

Utilização de namespaces

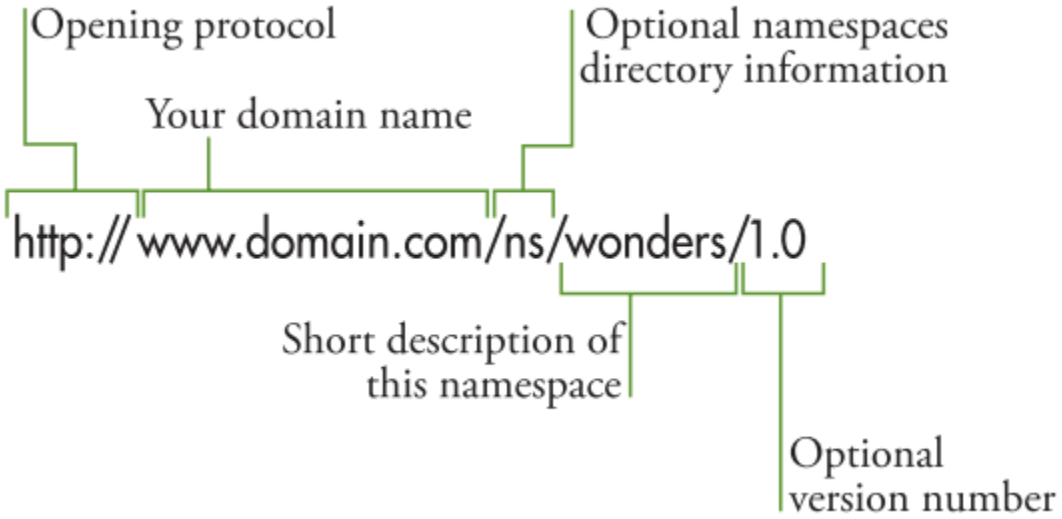
- Este formato permite a distinção dos elementos XML de cada um dos grupos;
- Se por exemplo identificar-mos um determinado “espaço” como ESTG, então (por exemplo) o tipo (ou elemento): ESTG.nome do exemplo anterior, não pode ser confundido com nenhum outro elemento de outro “espaço”.

Utilização de namespaces

- Normalmente não utilizamos nomes como ESTG para nos referirmos a um namespace, mas devemos utilizar nomes mais estruturados para o efeito;
- Em XML, os namespaces são escritos na forma de URI (Uniform Resource Identifier).

Utilização de namespaces

- Estrutura de um URI



Default Namespace

- Após a definição do nome do namespace, podemos declarar o namespace por defeito;
- Exemplo:

```
<?xml version="1.0" encoding="UTF-8"?>
<pizza xmlns="http:\\estg.ipp.pt\eoi\2020\Pizza">
  <name>Portuguesa</name>
  <ingredients>
    <ingredient>
      <name>Tomate</name>
      <category>Vegetal</category>
      <unit>grams</unit>
      <calories>200</calories>
    </ingredient>
  </ingredients>
  <price>20</price>
</pizza>
```

Definição de prefixos

- Na definição do namespace podemos utilizar um identificador para nos referirmos a um conjunto de elementos com um determinado namespace;
- Para isso, podemos definir um prefixo para um namespace e então utilizar o prefixo para referenciar elementos individuais;
- Os prefixos são definidos através de: xmlns:prefix, onde o prefix é o “nickname” para o namespace.

Definição de prefixos

- Após a declaração do prefixo para um namespace, podemos associar elementos para diferentes namespaces no XML;
- Recorremos a prefixos para identificar elementos de diferentes namespaces;
- Neste caso, especificamos o target namespace (namespace de destino) para o qual os elementos e atributos vão pertencer.

Definição de prefixos

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
        xmlns="http://www.aTascadaESTG.pt/Ingredient"
        targetNamespace="http://www.aTascadaESTG.pt/Ingredient"
        elementFormDefault="qualified">
(…)
```

Definição de prefixos

Ingrediente.XSD

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
        xmlns="http://www.aTascadaESTG.pt/Ingrediente"
targetNamespace="http://www.aTascadaESTG.pt/Ingrediente">
(...)
```

Pizza.XSD

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
        xmlns="http://www.aTascadaESTG.pt/Pizza"
        targetNamespace="http://www.aTascadaESTG.pt/Pizza"
        xmlns:c="http://www.aTascadaESTG.pt/Ingrediente"
        elementFormDefault="qualified">
(...)
<xs:element name="pizza">
        <xs:complexType>
          <xs:sequence>
            <xs:element name="nome" type="name" />
            <xs:element name="ingrediente">
             <xs:complexType>
              <xs:sequence>
                <xs:element name="nome" type="c:tipoNome" />
                (...)
```

Elementos qualificados

Utilização de prefixos

Importar XSD com namespace

- Os namespaces são um mecanismo muito útil para modularizar XML schemas, principalmente quando dividimos a nossa especificação em vários documentos;
- A divisão da especificação XSD em múltiplos ficheiros com namespaces bem definidos, permite-nos reutilizar as nossas definições em diferentes projetos, tornando o vocabulário mais fácil de ler modificar, uma vez que trabalhamos com pequenas partes do vocabulário;

Importar XSD com namespace

Ingrediente.XSD

```
<xs:schema   xmlns:xs="http://www.w3.org/2001/XMLSchema"
 xmlns="http://www.aTascadaESTG.pt/Ingrediente"
 targetNamespace="http://www.aTascadaESTG.pt/Ingrediente">
(...)
```

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
        xmlns="http://www.aTascadaESTG.pt/Pizza"
        targetNamespace="http://www.aTascadaESTG.pt/Pizza"
        xmlns:c="http://www.aTascadaESTG.pt/Ingrediente"
        elementFormDefault="qualified">
      <xs:import schemaLocation="Ingrediente.xsd"
namespace="http://www.aTascadaESTG.pt/Ingrediente"/>
(...)
<xs:element name="pizza">
        <xs:complexType>
          <xs:sequence>
            <xs:element name="nome" type="name" />
            <xs:element name="ingrediente">
             <xs:complexType>
               <xs:sequence>
                 <xs:element name="nome" type="c:tipoNome" />
               (...)
```

Com a inclusão de namespaces, o elemento import deverá ser utilizado para importar documentos XSD (ao invés do elemento include)

Associação XSD a XML

- xmlns – Descreve o namespace por defeito, indicando que todos os elementos utilizados neste documento estão declarados no namespace;
- xsi:schemaLocation – Este atributo possui dois valores: o nome do namespace a utilizar, ("Espaço"), e o caminho para o XSD utilizado nesse namespace;

Associação XSD a XML

Pizza.xml (excerto)

```
<?xml version="1.0" encoding="UTF-8"?>
<pizza xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns="http://www.aTascadaESTG.pt/Pizza"
    xsi:schemaLocation="http://www.aTascadaESTG.pt/Pizza Pizza.xsd">
        (...)
</pizza>
```

Associação XSD a XML

- O documento XML deverá possuir a identificação dos namespaces envolvidos no XSD utilizado para validação;
- Deve também ser definido um prefixo para cada um desses namespaces (elementFormDefault = “qualified”), com exceção do namespace que seja considerado o default namespace (se for utilizado).

Associação XSD a XML

- Exemplo:

  Pizza.xml (excerto)

```
<?xml version="1.0" encoding="UTF-8"?>
<pizza xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation=http://www.estg.ipp.pt/eoi/2020/pizzaria/pizzaTypes PizzaTypes.xsd
xmlns=http://www.estg.ipp.pt/eoi/2020/pizzaria/pizzaTypes  xmlns:i= "http://www.estg.ipp.pt/eoi/2020/pizzaria/ingredientTypes">
  <name>Portuguesa</name>
  <ingredients>
    <i:ingredient>
      <i:name>Tomate</i:name>
      <i:category>Vegetal</i:category>
      <i:unit>grams</i:unit>
      <i:calories>200</i:calories>
    </i:ingredient>
  </ingredients>
  <price>20</price>
</pizza>
```

Definição do prefixo i para o namespace

Utilização do prefixo

Bibliografia/referências

- Referências Web:
  - https://www.w3schools.com/xml/schema_intro.asp
  - https://www.intertech.com/Blog/xml-schema-elementformdefault-and-attributeformdefault/
  - https://www.liquid-technologies.com/xml-schema-tutorial/xsd-namespaces
- Livro:
  - Anders M. and Michel S., An introduction to XML and Web Technologies, Addison-Wesley, 2006;
  - Eito-Brun, R. XML-based Content Management: Integration, Methodologies, and Tools (1st ed.). Chandos Publishing, 2017;

P.PORTO

ESCOLA SUPERIOR DE TECNOLOGIA E GESTÃO

XML Schema

Bruno Oliveira (bmo@estg.ipp.pt)

PEI Processamento Estruturado de Informação